

Weekly Rapport — Weeks 4–5

Real-Time Anomaly Detection Platform for Branch Teller Operations

Yassine — Amen Bank, Direction Centrale du Système d’Information

June 18–24, 2026

Contents

1	Executive Summary	1
2	Model Training — Initial Attempt (June 18–19)	2
2.1	Autoencoder	2
2.2	LSTM	2
3	Root Cause Analysis and Pipeline Rebuild (June 20–21)	2
3.1	Diagnosis	2
3.2	Redesign Decisions	2
3.3	Rebuilt Results	3
4	Score Fusion Layer (June 22)	3
4.1	Problem	3
4.2	Design	3
5	Decision Service (June 22)	4
5.1	Architecture	4
5.2	Results	4
6	Batch Pipeline Realignment (June 24)	4
6.1	Problem	4
6.2	Fixes Applied	4
7	Key Technical Learnings	5
8	Artifacts Produced	5
9	Next Steps	6

1 Executive Summary

This rapport covers the most intensive development period of the project (June 18–24), during which the ML pipeline went from initial model training through a full diagnostic rebuild to a complete end-to-end detection system. The week began with model training that revealed structural issues

in the feature pipeline, led to a complete enrichment redesign, and concluded with a working Score Fusion layer, Decision Service, and a clean PostgreSQL cold store aligned with the rebuilt pipeline.

Key metrics at week’s end: Autoencoder test AUC 0.9853, LSTM test AUC 0.9866, fused detection rate 87.5% at REVIEW tier, BLOCK precision 80.4%.

2 Model Training — Initial Attempt (June 18–19)

2.1 Autoencoder

The Autoencoder was trained on normal-only enriched events using the 74-feature schema produced by the original batch enrichment pipeline. Three experiment configurations were evaluated using Optuna with AUC-ROC as the objective (chosen over F-beta due to the extreme class imbalance at 0.087% anomaly rate):

- 71 features with mean MSE scoring: AUC 0.577
- 34 behavioral features with pre-computed signals removed: AUC 0.41
- 71 features with max/top-k scoring: AUC 0.557

2.2 LSTM

The LSTM used a multi-task prediction architecture with 8 heads: operation type classification, amount bucket prediction (8 buckets with a dedicated 8,000–10,000 DT smurfing band), and binary flag prediction for anomaly-correlated features. After Optuna search: AUC 0.564.

Both models performed near random, triggering a root cause investigation.

3 Root Cause Analysis and Pipeline Rebuild (June 20–21)

3.1 Diagnosis

Two independent root causes were identified:

1. **Undetectable scenarios:** Many of the original 13 anomaly scenarios either had stub implementations or did not produce measurable changes in the enriched feature space. For instance, `timing_anomaly` shifted the event hour, but the enriched event contained no per-event hour feature — only a profile-level hour distribution that absorbs single-event changes.
2. **Profile dominance:** The 74-feature enriched event was dominated by raw profile field dumps (weekly snapshot means, stds, distributions) that were identical across all events from the same client in the same week. Per-event signal was drowned out by profile noise. This was identified by Yassine during analysis: the profile was dominating the raw event in a way that made the enriched events nearly indistinguishable between normal and anomalous.

3.2 Redesign Decisions

- **Generator:** Reduced from 13 to 9 well-implemented scenarios with verified feature-space footprints. Three difficulty tiers preserved: easy (`amount_spike`, `round_amount`, `duplicate_virement`), medium (`frequency_burst`, `cumulative_threshold`, `timing_anomaly`), hard (`smurfing`, `cheque_structuring`, `repeated_client_employee`).

- **Enrichment:** Complete redesign from profile dumps to **contrast features** — each feature measures the deviation of the current event against the client’s baseline rather than repeating the baseline itself. Feature count dropped from 74 to 30, organized as: 7 raw event features, 17 contrast features (z_amount, amount_to_mean_ratio, threshold flags, op_type_probability, timing contrasts, counterparty contrast, velocity metrics), and 6 context features (archetype one-hot encoding, account_age_days).
- **Oracle profile shortcut:** Instead of weekly snapshot profiles, client personality parameters are exported directly from the generator’s in-memory objects to `clients.csv`. This provides the exact baseline needed for contrast computation without the weekly granularity limitation. The oracle shortcut will be replaced by batch-computed profiles in the production streaming pipeline.

3.3 Rebuilt Results

Model	Old AUC	Rebuilt AUC
Autoencoder	0.577	0.9935
LSTM	0.564	0.9430

The improvement is not from better hyperparameters — it is entirely structural. Contrast features expose per-event deviations that reconstruction-based models can detect.

4 Score Fusion Layer (June 22)

4.1 Problem

The Autoencoder and LSTM produce anomaly scores on fundamentally different scales (AE: MSE reconstruction error, LSTM: prediction error). Raw score combination is meaningless without normalization.

4.2 Design

The fusion mechanism uses three components:

1. **Percentile normalization:** Each model’s raw score is mapped to a percentile rank against a saved reference distribution computed from normal training data. This places both scores on a common $[0, 1]$ scale where 0.95 means “higher than 95% of normal events.”
2. **Maturity-adaptive weighting:** A sigmoid function of `days_since_account_opening` (midpoint 90 days, steepness 15) controls the weight between models. New accounts favor the Autoencoder (works from the first event), while mature accounts favor the LSTM (requires sequence history). `days_since_account_opening` was chosen over event count because event count penalizes low-frequency archetypes like retirees.
3. **Weighted combination:** $S_{fused} = w_{lstm} \cdot S_{lstm} + (1 - w_{lstm}) \cdot S_{ae}$

Fused AUC on validation set: **0.9725**.

5 Decision Service (June 22)

5.1 Architecture

The Decision Service is a stateless three-layer engine:

Layer 1 — Score thresholds: Calibrated on the validation set.

Tier	Threshold	Meaning
REVIEW	0.95	Supervisor attention needed
ALERT	0.99	Requires action
BLOCK	0.999	Transaction should be blocked

Layer 2 — Regulatory rules: Eight rules that can escalate tiers (never de-escalate). Key rules include:

- BCT 10,000 DT reporting threshold (with $z_amount \geq 2$ guard to prevent routine big-business transactions from flooding ALERT queue)
- Smurfing band detection (8,000–9,999 DT with velocity check)
- Passeur de fonds indicator (same employee-client pair ≥ 3 in 24h)
- Cheque ceiling proximity (25,000–30,000 DT band)
- Frequency burst ($> 3\times$ expected daily rate)

All regulatory flags are logged for audit regardless of whether they trigger an escalation.

Layer 3 — Risk history adjustment: Recidivist escalation (prior alerts boost tier), false-positive dampening (repeated supervisor rejections reduce sensitivity), and cold-start conservatism (new accounts get a more aggressive scoring posture).

5.2 Results

- Detection rate at REVIEW tier: 93.7%
- BLOCK precision: 71%
- Alerts per branch per day: ~ 1.7 at REVIEW threshold

6 Batch Pipeline Realignment (June 24)

6.1 Problem

The PostgreSQL cold store, batch profile scripts, and data loading were built before the generator rebuild. All stored data was from the old run with different client IDs, different transaction patterns, and a mismatched schema (including an `initial_balance` column no longer needed).

Additionally, the generator had a latent bug: employees were assigned from a global pool rather than being branch-linked, allowing the same employee to appear at multiple branches — a physical impossibility in branch banking.

6.2 Fixes Applied

1. **PostgreSQL schema rebuilt from scratch:** All tables dropped and recreated. `initial_balance` removed from `clients_master`. Alert severity tier changed from

VARCHAR to PostgreSQL ENUM type for database-level validation.

2. **Generator fixed:** Employee pool restructured as a branch-linked dictionary (2 employees per branch). Normal event generation picks employees from the visited branch. The `repeated_client_employee` anomaly scenario now forces both the employee and branch as a pair, reflecting the real-world pattern of a client repeatedly visiting a specific branch for a specific teller.
3. **Data regenerated:** 243,904 events, 10,000 clients, 300 employees (150 branches $\times$ 2). Validated: 0 employees appearing at multiple branches.
4. **Models retrained on fresh data:**

Model	Pre-fix AUC	Post-fix AUC
Autoencoder	0.9935	0.9853
LSTM	0.9430	0.9866

The LSTM improved substantially — branch-linked employees produce cleaner sequential patterns. The slight AE drop is expected variance from the different data.

5. **Threshold recalibration:** Same thresholds (T1=0.95, T2=0.99, T3=0.999) validated on new data. 78 alerts/day across 150 branches ($\approx$ 0.5 per branch per day) — manageable for supervisors.
6. **Batch profiles recomputed:** `profile_snapshots`, `employee_profile_snapshots`, and `archetype_baselines` repopulated from fresh PostgreSQL data.

7 Key Technical Learnings

1. **Contrast features over profile dumps:** Features should measure event-vs-baseline deviation, not repeat the baseline. Profile dominance drowns per-event signal and makes anomalous events look identical to normal ones.
2. **Training-inference symmetry:** Models must train on exactly the feature schema they will see in production. The oracle shortcut produces the same 30 features as the streaming Enrichment Service will.
3. **Branch-linked employees matter:** A global employee pool creates impossible data (one teller at multiple branches). This isn't just data quality — it contaminates the `repeated_client_employee` scenario signal.
4. **Maturity proxy choice matters:** Event count as a maturity signal penalizes low-frequency archetypes (retirees transact rarely but are not “new”). `days_since_account_opening` is the correct signal.
5. **Rules escalate only:** Regulatory rules can raise alert tiers but never lower them. All regulatory flags are audit-logged regardless of escalation.

8 Artifacts Produced

File	Description
src/generator/generator.py	Rebuilt generator with branch-linked employees, 9 scenarios
src/generator/client.py	Client class with account_opening_date
src/enrichment.py	Contrast-focused enrichment (30 features)
src/scoring/fusion.py	Score fusion: percentile normalization + sigmoid maturity weight
src/decision/decision.py	3-layer Decision Service with 8 regulatory rules
src/training/data_prep.py	Data preparation with warm-up filter and temporal split
src/training/autoencoder.py	Autoencoder training and evaluation
src/training/lstm.py	LSTM training and evaluation
scripts/compute_reference_scores.py	Compute reference score distributions for fusion
scripts/calibrate_thresholds.py	Threshold analysis on validation set
scripts/load_postgresql.py	One-time PostgreSQL data loading
models/	AE + LSTM models, scaler, feature_cols, reference distributions

9 Next Steps

1. **SHAP integration** into the Scoring Service for per-alert explainability
2. **Streaming pipeline** via RedPanda (Enrichment → Scoring → Decision services)
3. **Streamlit dashboard** consuming the decisions topic
4. **MLflow + Airflow** for model versioning and orchestrated retraining
5. **Batch enrichment rewrite** (aligned with 30-feature schema, triggered by Airflow DAG)